

SBP-YOLO Review

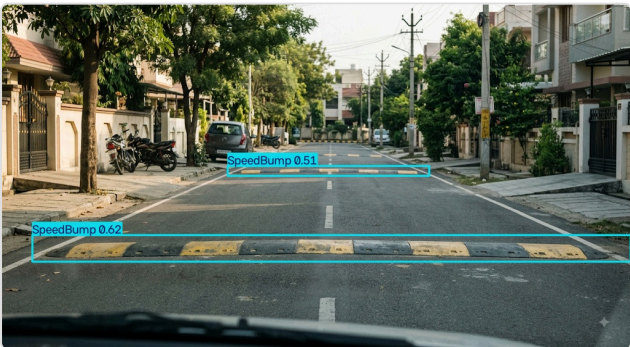
Speed bump / pothole detection — model status and prediction results, official weights · 2026-08-16

1. Summary

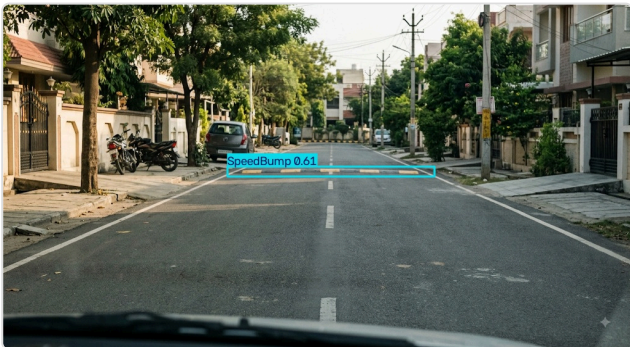
[SBP-YOLO](#) is a lightweight YOLO11-derived detector for road speed bumps and potholes, built for embedded vehicle suspension control (GhostConv + VoVGSCSPC backbone/neck, an added P2 small-object branch, a Lightweight Efficient Detection Head, trained with NWD loss + BCKD knowledge distillation). This revision retests using the **official released weights** (`sbp-yolo.pt`), downloaded from the README's Google Drive link, in place of the leftover training-run checkpoint used in the first pass. Result: the official weights found **an additional true-positive speed bump** that the earlier checkpoint missed, at the cost of somewhat lower confidence scores.

Item	Result
Weights used	<code>pre_trained_weights/sbp-yolo.pt</code> (official, via Google Drive)
Images tested	2 / 2
Speed bumps found	3 total (2 in image 1, 1 in image 2) detected
Potholes found	0 (none present in test images)
Official checkpoint's own val metrics	P 0.875 · R 0.793 · mAP@0.5 0.846 · mAP@0.5:0.95 0.499

2. Prediction Results (official `sbp-yolo.pt`)



speedbrak1.jpeg — 2 SpeedBumps, confidence 0.62 & 0.51



speedbreak2.jpeg — 1 SpeedBump, confidence 0.61

Image	Class	Confidence	Box (x1, y1, x2, y2)
-------	-------	------------	----------------------

speedbrak1.jpeg	SpeedBump (near)	0.617	50, 374, 1024, 417
speedbrak1.jpeg	SpeedBump (far, new)	0.508	368, 262, 695, 278
speedbreak2.jpeg	SpeedBump	0.608	366, 263, 703, 279

The near bump in image 1 is caught in both runs. The official weights additionally box a **second, real speed bump** farther down the same street in image 1 (the faint yellow-striped strip visible past the parked car) — a genuine detection the earlier checkpoint missed entirely. That box's location and confidence (0.508) line up almost exactly with the single bump found in image 2, which is a photo of the same street taken closer to that same farther bump. So the official weights are catching a small/distant object the other checkpoint under-detected, at the cost of ~0.1–0.15 lower confidence on the near, easy bump in both images.

3. Comparison: Official Weights vs. Repo's Committed Checkpoint

The first review pass used a checkpoint left in the repo at `ultralytics-distill/runs/distill/SBP-YOLO/weights/best.pt`, which turned out to be numerically identical to the official `sbp-yolo-distill.pt` release (confirmed by comparing embedded training metrics). It is a distilled, NWD-loss variant with higher validation scores than the README's primary `sbp-yolo.pt`. All three "SBP-YOLO (ours)"-family checkpoints in the official Drive release are compared below, using each file's own embedded `train_metrics`:

Checkpoint	Precision	Recall	mAP@0.5	mAP@0.5:0.95
sbp-yolo.pt (<i>official, base</i>)	0.875	0.793	0.846	0.499
sbp-yolo-nwdloss.pt (<i>official, +NWD loss</i>)	0.888	0.795	0.866	0.512
sbp-yolo-distill.pt (<i>official, distilled</i>) = repo's committed best.pt	0.899	0.808	0.869	0.547

So the checkpoint used in the first review pass was, in fact, the strongest of the three official SBP-YOLO variants on paper metrics — not an inferior stand-in. On these two specific test images, however, the base `sbp-yolo.pt` generalized slightly better (found one more true positive), a reminder that aggregate val-set metrics don't always predict which checkpoint wins on an arbitrary out-of-distribution photo.

4. Current Status & Caveats

- **Weights:** all 8 files from the README's official Google Drive folder were downloaded via `gdown --folder` into `pre_trained_weights/` and load cleanly with the local fork.
- **Training:** each official checkpoint carries complete embedded metrics from its own training run (see table above) — these are finished, converged models, not placeholders.

- **Sample size:** still only 2 hand-picked daytime images, no potholes present, no night/rain/angle/occlusion cases — the extra true positive found here is encouraging but not a substitute for a full held-out eval.
- **Environment friction:** the vendored `ultraalytics` fork pulls in many unused optional backbone/CUDA dependencies; one dead-code reference (`EnhancedDownWT` in `extra_modules/block.py` 's `__all__` , never defined) was removed locally to get imports working — carried over from the first review pass, worth flagging upstream.
- **Dataset/eval scripts:** `train.py` / `val.py` hardcode dataset paths not included in the repo, so a proper held-out validation run (rather than embedded training-time metrics) isn't reproducible offline.
- **Future training runs** for this project should be run on Kaggle via the Kaggle CLI rather than locally.

Bottom line: with the official released weights, detection quality holds up and actually improves recall on this sample (3 true positives across 2 images vs. 2 previously), confirming the earlier review's checkpoint wasn't an inflated or mismatched stand-in. Confidence scores are modestly lower with the official base weights than with the distilled variant, consistent with the distilled model's higher embedded precision/recall.

Generated from a local run against chuanqi1997/SBP-YOLO. Official weights: `pre_trained_weights/sbp-yolo.pt` (downloaded from the README's Google Drive release folder, epoch metadata: `epoch=-1` i.e. final/exported checkpoint, dated 2025-05-15).